

UNIODONTO PASSOS

Sistema de Dashboard de Gestão

Documentação Oficial

Versão 1.2 — Junho 2026

Produto	Dashboard de KPIs, Relatórios e Gestão
Cliente	Uniodonto Passos — Cooperativa Odontológica
Versão do Sistema	1.2
Stack Principal	React 19 · TypeScript · Vite · Tailwind CSS v4
Plataformas	Web (Firebase Hosting) · Android (Capacitor)
Data do Documento	01/06/2026
Autor	FerTaise Tech
Classificação	Confidencial — Uso Interno

Sumário

1. Visão Geral do Produto
2. Arquitetura Técnica
 - 2.1 Stack e Dependências
 - 2.2 Estrutura de Pastas
 - 2.3 Fluxo de Dados
3. Páginas e Funcionalidades
 - 3.1 Dashboard Principal
 - 3.2 Relatórios
 - 3.3 Envio de Dados
 - 3.4 Configurações
 - 3.5 Login / Autenticação
4. Componentes de Interface
 - 4.1 Layout e Navegação
 - 4.2 KPI Cards
 - 4.3 Gráficos
 - 4.4 Tabelas
 - 4.5 Mobile
5. Gerenciamento de Estado
6. Utilitários e Serviços
 - 6.1 Adaptador de Dados
 - 6.2 Cálculos e Métricas
 - 6.3 Exportação PDF e CSV
 - 6.4 Parser de Planilhas
 - 6.5 Pipeline de APIs
7. Dados Mock e Tipos TypeScript
8. Testes
9. Deploy e Infraestrutura
10. Design System
11. Usuários e Segurança
12. Roadmap e Pendências

1. Visão Geral do Produto

O **Dashboard Uniodonto Passos** é uma aplicação web/mobile SPA (Single Page Application) desenvolvida para centralizar a gestão operacional e estratégica da cooperativa odontológica Uniodonto Passos. O sistema consolida KPIs de beneficiários, desempenho de campanhas de marketing digital, análise de investimentos e geração de relatórios gerenciais em PDF.

Objetivos

- Substituir planilhas dispersas por um painel centralizado e responsivo.
- Oferecer visibilidade em tempo real dos indicadores de saúde da cooperativa.
- Automatizar a geração de relatórios executivos para a diretoria.
- Permitir upload de dados via planilhas Excel e sincronismo com APIs de anúncios.
- Suportar acesso mobile nativo via Android (Capacitor).

Público-Alvo

Perfil	Descrição	Acesso Principal
Diretores	Dr. Elcio, Dr. Luiz Fernando, Dr. Mateus José	Dashboard, Relatórios
Gerência	Janaína Pádua	Dashboard, Envio de Dados, Relatórios
Administrativo TI	FerTaise Tech Admin	Todas as áreas + Configurações

Principais Funcionalidades

Funcionalidade	Descrição
Dashboard KPIs	6 cartões dinâmicos: Beneficiários, Leads, Conversões, Investimento, ROI e NPS
Filtros de Período	Navegação mensal com histórico de 6 meses (Jan–Jun 2026)
Gráficos Interativos	Desempenho de anúncios (Chart.js) e funil de conversão geométrico
Tabela de Investimentos	14 categorias, filtros por tipo, totalizações dinâmicas
Relatórios PDF	5 tipos de relatório com análise estratégica (jsPDF + autoTable)
Upload Excel	Parser SheetJS com validação estrita e mapeamento multicolumnas
Sincronismo APIs	Pipeline simulado de Google Ads e Meta ADS
Gerenciamento de Usuários	CRUD completo em Settings com autenticação local
Tema Dark/Light	Persistido em localStorage
Mobile Nativo	Capacitor Android com bottom navigation e layout responsivo

2. Arquitetura Técnica

2.1 Stack e Dependências

O projeto adota uma stack moderna de front-end, sem backend próprio. A persistência é feita via **localStorage** e o deploy via **Firebase Hosting**.

Camada	Biblioteca / Ferramenta	Versão	Finalidade
UI Framework	React	19.2.6	Componentes, hooks, contexto
Linguagem	TypeScript	6.0.3	Tipagem estática e segurança
Build Tool	Vite	8.0.14	Bundling rápido, dev server porta 3080
Estilização	Tailwind CSS	4.3.0	Utility-first, dark mode, breakpoints
Gráficos	Chart.js + react-chartjs-2	4.5.1 / 5.3.1	Barras, linhas, funil
PDF	jsPDF + jspdf-autotable	4.2.1 / 5.0.8	Geração de relatórios
Excel	XLSX (SheetJS)	0.18.5	Parser de planilhas
Ícones	Lucide React	1.17.0	SVG icons
Mobile	Capacitor	8.3.4	Wrapper Android nativo
Testes	Vitest + Testing Library	4.1.7 / 16.3.2	Unit/integration tests
Hospedagem	Firebase Hosting	—	CDN global, SPA rewrite rules

2.2 Estrutura de Pastas

Pasta / Arquivo	Descrição
src/App.tsx	Raiz da aplicação — routing via useState, tema, usuário logado
src/main.tsx	Entry point React — monta <App> no DOM
src/pages/	5 páginas: Dashboard, Reports, DataUpload, Login, Settings
src/components/layout/	Sidebar, Header, MobileBottomNav
src/components/cards/	6 KPI cards + KPICardGrid (carrossel)
src/components/charts/	AdPerformanceChart, ConversionFunnelTabs, TrendCharts
src/components/tables/	InvestmentTable (desktop), MobileInvestmentList
src/components/mobile/	7 componentes exclusivos mobile
src/context/DashboardContext.tsx	Estado global via Context API
src/hooks/useDashboard.ts	Hook com exportação CSV
src/utills/	8 utilitários: adapter, calculations, formatters, validator, parser, pdfExporter, adApiPipeline, route
src/data/	mockDashboardData, mockInvestments, mockReports
src/types/	Interfaces TypeScript: dashboard, reports, investments
src/test/	7 arquivos de teste

android/	Projeto Capacitor Android gerado
dist/	Build otimizado (saída Vite) para deploy
firebase.json / .firebaserc	Config Firebase Hosting (projeto: uni0dontopassos)
capacitor.config.ts	Config Capacitor (App ID: com.dashboard.app)
vite.config.ts	Dev port 3080, alias @→src, vitest jsdom
tailwind.config.js	Cores brand, dark mode, breakpoints customizados

2.3 Fluxo de Dados

O sistema segue um fluxo unidirecional: dados mock (ou uploaded) são armazenados no DashboardContext, adaptados para o formato legado dos componentes via dashboardAdapter, e renderizados nos componentes de UI. Interações do usuário atualizam o contexto via setState, que persiste no localStorage automaticamente.

Etapa	Módulo	Ação
1 - Origem	mockDashboardData.ts / upload	Dados de 6 meses ou planilha Excel carregada
2 - Contexto	DashboardContext.tsx	Armazena allDashboardData + investmentsData no state
3 - Seleção	MonthNavigator	Usuário seleciona mês → currentMonthData atualizado
4 - Adaptação	dashboardAdapter.ts	adaptModernToLegacy() converte payload para formato UI
5 - Cálculos	dashboardCalculations.ts	CAC, LTV, Churn, taxa de conversão calculados
6 - Render	Componentes (cards, charts, tables)	Recebem dados adaptados e renderizam
7 - Persistência	localStorage	Estado salvo com versão v1.5 para cache automático
8 - Export	pdfExporter.ts / useDashboard.ts	PDF ou CSV gerado e baixado pelo usuário

3. Páginas e Funcionalidades

3.1 Dashboard Principal

Página central do sistema, acessível após login. Exibe o painel completo de KPIs, gráficos de desempenho e tabela de investimentos. Layout totalmente responsivo com 3 modos de exibição: desktop (12 colunas), tablet e mobile.

Área	Componentes
KPIs (filtro Geral)	BeneficiariosCard, LeadsCard, ConversoesCard, InvestimentoCard, RoiCard, NpsCard
KPIs (filtro Marketing)	Cards reorganizados para foco em Leads, Conversões e ROI de campanhas
KPIs (filtro Análise)	Cards com métricas de crescimento, churn e LTV
Gráfico de Anúncios	AdPerformanceChart — barras semanais por plataforma (Google/Meta/Instagram)
Funil de Conversão	ConversionFunnelTabs — 3 abas: Funil, Origens, Cidades
Investimentos Desktop	InvestmentTable — 5 filtros de categoria, totalizações
Investimentos Mobile	MobileInvestmentList — cards verticais com mesmos filtros

3.2 Relatórios

Página de geração de relatórios consolidados com filtros de período e tipo. Suporta 5 tipos de relatório, exportação em PDF e CSV.

Tipo de Relatório	Conteúdo
Executivo	Resumo completo para diretoria — KPIs, tendências, análise estratégica CEO
Comercial	Foco em vendas — conversões, CAC, CPL, pipeline de leads por canal
Churn / Retenção	Análise de cancelamentos, taxa de churn, identificação de padrões
Financeiro	Investimentos por categoria, ROI, LTV, break-even
Satisfação NPS	Score NPS, promotores, detratores, comparativo histórico

3.3 Envio de Dados

Centraliza todas as formas de atualização de dados. Possui 4 sub-abas de navegação.

Resumo (envio):

Visão geral das fontes de dados e status de sincronismo.

Envio Manual:

Formulário para digitação direta de KPIs mensais.

Planilhas:

Upload de arquivo Excel (.xlsx) com validação estrita e mapeamento automático de colunas.

Conexões:

Pipeline simulado de sincronismo com Google Ads API e Meta ADS API, com logs de progresso.

3.4 Configurações

Painel administrativo completo para gestão de usuários e preferências do sistema.

- CRUD de usuários: criar, editar, excluir, redefinir senha.
- Alternância de tema (dark/light) persistido em localStorage.
- Gerenciamento de sessão e informações de conta.
- Configurações de segurança (redefinição de senha).

3.5 Login / Autenticação

Autenticação local sem backend. Credenciais armazenadas em localStorage (**uniodonto_settings_users**). Sem sessão persistente entre abas — ao recarregar, usuário precisa autenticar novamente.

Nome	Email	Role	Senha Padrão
FerTaise Tech Admin	fertaisetech@gmail.com	Tech FerTaise	1234
Dr. Elcio Beraldo	elcio@uniodonto.com	Diretor	1234
Dr. Luiz Fernando	luiz@uniodonto.com	Diretor	1234
Dr. Mateus José	mateus@uniodonto.com	Diretor	1234
Janaína Pádua	gerente@uniodonto.com	Gerente	1234

4. Componentes de Interface

4.1 Layout e Navegação

Componente	Arquivo	Descrição
Sidebar	components/layout/Sidebar.tsx	Menu lateral colapsável. Estado persistido em localStorage. Modal de ajuda com links para documentação.
Header	components/layout/Header.tsx	Título dinâmico conforme filtro ativo. FilterTabs (Geral/Marketing/Análise). Modo de tela (md, lg, xl).
MobileBottomNav	components/layout/MobileBottomNav.tsx	Bottom navigation com 4 ícones. Visível apenas em mobile (<md). Área tátil para navegação rápida.
MonthNavigator	components/navigation/MonthNavigator.tsx	Seletor de período mensal com botões prev/next e label 'Maio/2026'.
FilterTabs	components/filters/FilterTabs.tsx	3 abas de filtro para área de visualização do dashboard.

4.2 KPI Cards

Card	Métricas Exibidas
BeneficiariosCard	Total, % vs mês anterior, ativos/novos/cancelados, gráfico rosca PF×PJ
LeadsCard	Total leads, breakdown por origem (Google Ads, Meta, Indicação, Outros)
ConversoesCard	Taxa de conversão %, vendas realizadas, total leads, progresso vs meta
InvestimentoCard	Total investido, % do orçado, barra de progresso, orçado vs real
RoiCard	ROI total, CAC (custo aquisição cliente), LTV, fator LTV/CAC
NpsCard	Score NPS, classificação (Excelente/Bom/Neutro), promotores vs detratores
KPICardGrid	Container: carrossel horizontal em mobile/tablet, grid em desktop

4.3 Gráficos

Componente	Tecnologia	Descrição
AdPerformanceChart	Chart.js	Barras semanais (7 dias) por plataforma. 6 métricas: views, grupos, investido, leads, conversões, ROI.
ConversionFunnelTabs	CSS clip-path	3 abas: Funil geométrico (5 etapas: Impressões→Cliques→Leads→Agendamentos→Verificações).
TrendCharts	Chart.js	8 séries temporais para relatórios consolidados. Usado na página Reports.

4.4 Tabelas

Componente	Modo	Funcionalidades
InvestmentTable	Desktop (md+)	14 itens, 5 filtros (Todos/Marketing/Ads/Offline/Ferramentas), tags coloridas por categoria.
MobileInvestmentList	Mobile (<md)	Cards verticais verticais com mesmo filtro de categoria e totalizações.

4.5 Componentes Mobile Exclusivos

Componente	Descrição
------------	-----------

MobileDashboardHeader	Header adaptado para mobile com título e controles compactos
MobileDashboardTabs	Abas de navegação de seções do dashboard em mobile
MobileKpiStrip	Strip horizontal de KPIs scrollável para mobile
MobileBeneficiariesCard	Card de beneficiários otimizado para tela pequena
MobileAdsPerformanceCard	Card de desempenho de anúncios para mobile
MobileInvestmentsPreviewCard	Preview compacto de investimentos
MobileFunnelCard	Funil de conversão simplificado para mobile

5. Gerenciamento de Estado

O sistema utiliza **React Context API** como solução de estado global, sem Redux ou Zustand. O contexto principal é o **DashboardContext**, provedor único no topo da árvore de componentes.

DashboardContext — Interface

Propriedade	Tipo	Descrição
selectedMonth	string	Mês selecionado no formato YYYY-MM (ex.: '2026-05')
setSelectedMonth	function	Atualiza o mês selecionado e recalcula currentMonthData
availableMonths	array	Lista de meses com valor e label formatado para o seletor
currentMonthData	DashboardDataPayload	Dados completos do mês atualmente selecionado
allDashboardData	MonthDataMap	Mapa indexado por mês com todos os dados históricos
investmentsData	InvestmentPayload	Dados de investimentos (categorias + detalhes mensais)
reportFilter	ReportFilter	Filtros aplicados aos relatórios (tipo, período)
consolidatedReport	ConsolidatedReport	Relatório gerado dinamicamente em tempo real
resetToDefaultData	function	Restaura dados mock originais e limpa localStorage
upsertMonthData	function	Insere ou atualiza dados de um mês específico

Persistência — localStorage

Chave	Conteúdo
uniodonto_dashboard_data	allDashboardData (todos os meses em JSON)
uniodonto_investments_data	investmentsData (categorias e valores mensais)
uniodonto_dashboard_version	Versão atual: 'v1.5' — limpa cache ao atualizar
uniodonto_settings_users	Lista de usuários cadastrados no sistema
sidebar-collapsed	Estado de collapse da Sidebar ('true'/'false')
theme	Tema ativo ('light' ou 'dark')

Estado em App.tsx

Além do DashboardContext, o componente raiz **App.tsx** mantém estado local para:

- currentPage (string) — rota atual da aplicação (sem react-router).
- theme ('light' | 'dark') — aplicado como classe no elemento raiz.
- loggedUser (User | null) — usuário autenticado, null se não logado.

6. Utilitários e Serviços

6.1 Adaptador de Dados — dashboardAdapter.ts

Converte dados no formato moderno (**DashboardDataPayload**) para o formato legado (**MonthDashboardData**) esperado pelos componentes de UI. Usa sub-mapeadores especializados em **subMappers.ts**.

Sub-mapeador	Saída
mapBeneficiarios()	BeneficiariosData — total, ativos, novos, cancelados, PF, PJ
mapLeads()	LeadsData — total, google, meta, indicacao, outros
mapConversoes()	ConversoesData — taxa, vendas, leads, meta
mapInvestimento()	InvestimentoData — total, orcado, percentual
mapRoi()	RoiData — roi, cac, ltv, fatorLtv
mapNps()	NpsData — score, status, respostas, promotores, detratores
mapInvestimentosTabela()	InvestmentRow[] — lista completa para tabela
mapAnuncios()	AnunciosData — métricas semanais por plataforma
mapFunil()	FunilData — 5 etapas de conversão
mapCidades()	CidadesData[] — distribuição geográfica de leads

6.2 Cálculos e Métricas — dashboardCalculations.ts

Função	Fórmula / Lógica
calcularTaxaConversao(c, l)	conversoes / leads × 100
calcularCAC(total, novos)	totalInvestido / novosBeneficiarios
calcularChurn(cancel, ativos)	cancelados / beneficiariosAtivos × 100
calcularLTV(ticket, churn, ltv?)	ticketMedio / (churnRate/100) — ou inputLtv se fornecido
processarInvestimentos(inv, cats)	Agrupa e soma investimentos por tipo (marketing, sales, operational)
formatarMonthLabel(month)	'2026-05' → 'Maio/2026'

6.3 Formatters — formatters.ts

Função	Exemplo de Saída
formatarNumero(1234)	1.234
formatarMoeda(1234.56)	R\$ 1.234,56
formatarPorcentagem(12.5)	12,5%
formatarMilharMoeda(12500)	R\$ 12,5 mil

6.4 Exportação — pdfExporter.ts

Gera PDFs via **jsPDF + jspdf-autotable**. Função principal: **exportarPDF(consolidatedReport, filter, allDashboardData)**. Cores padronizadas (brand pink #D81B60), análises estratégicas dinâmicas, tabelas com totalizações.

6.5 Parser de Planilhas — spreadsheetParser.ts

Parser usando **SheetJS (XLSX)**. Mapeia abas: Resumo, Tráfego, Canais, Cidades, Campanhas, Investimentos. Busca de colunas flexível (case-insensitive, acentuação). Retorna **ParsedExcelData** estruturado. Inclui **downloadExcelTemplate()** para template de exemplo.

6.6 Pipeline de APIs — adApiPipeline.ts

Função **syncAdsApis(credentials, selectedMonth, callback)**. Simula sincronismo com Google Ads e Meta ADS. Pipeline com logs de progresso (**ApiProgressLog[]**). Retorna dados estruturados de campanhas por plataforma.

6.7 Validação — dataValidator.ts

Validação estrita de dados uploadados. Verifica: campos numéricos, ausência de negativos, consistência lógica (conversões $\leq$ leads). Emite **warnings** de negócio (volume zero, cancelamentos zero, divergências regionais). Inclui **safeDivide(n, d)** para evitar NaN/Infinity.

7. Dados Mock e Tipos TypeScript

Dados Mock — 6 Meses

O sistema opera com dados simulados de Janeiro a Junho de 2026, indexados por chave YYYY-MM em `mockDashboardData.ts`. Cada mês contém:

Campo	Tipo	Descrição
summary	MonthlySummary	13 KPIs principais (beneficiários, leads, conversões, ROI, NPS, etc.)
trafficSources[]	TrafficSource[]	4 origens: Google Ads, Meta ADS, Parcerias, Orgânico
acquisitionChannels[]	AcquisitionChannel[]	3 canais: Digital, Venda Direta, Corretores
cityDistribution[]	CityDistribution[]	5 cidades: Passos, Itaú, S.S. Paraíso, Cássia, Alpinópolis
campaigns[]	CampaignPerformance[]	4 campanhas com métricas completas por plataforma

Investimentos Mock

`mockInvestments.ts` — 14 categorias de investimento com histórico mensal.

Tipo	Total Mensal (Referência)
Marketing (offline + ferramentas)	R\$ 6.902,00
Sales (comissões, eventos)	R\$ 7.115,00
Operational	R\$ 0,00
TOTAL CONSOLIDADO	R\$ 14.017,46

Hierarquia de Tipos TypeScript

Namespace	Tipos Principais
Tipos Modernos (DashboardDataPayload)	MonthlySummary, TrafficSource, AcquisitionChannel, CityDistribution, CampaignPerformance
Tipos Legados (MonthDashboardData)	BeneficiariosData, LeadsData, ConversoesData, InvestimentoData, RoiData, NpsData
Tipos de Investimentos	InvestmentCategory, MonthlyCategoryInvestment, InvestmentPayload
Tipos de Relatórios	ReportType (5 valores), ConsolidatedReportRow, ConsolidatedReportTotals, ConsolidatedReport
Tipos Globais	MonthDataMap (index por YYYY-MM), User, Theme

8. Testes

O projeto usa **Vitest** (compatível com Jest) e **@testing-library/react**. Ambiente de teste: **jsdom**. Configuração em **src/vitest.setup.ts**.

Arquivo	Escopo	Principais Casos
test/Login.test.tsx	Página Login	Renderização, validação de credenciais, mensagem de erro, redirect após login
test/Settings.test.tsx	Página Settings	CRUD de usuários: criar, editar, excluir, redefinir senha, listagem
utils/dashboardAdapter.test.ts	dashboardAdapter	Conversão modern→legacy, valores nulos, meses sem dados
utils/dataMap.test.ts	Mapeamento KPIs	Mapeamento correto de todos os campos, edge cases numéricos
utils/InvestmentTable.test.tsx	InvestmentTable	Filtros de categoria, totalizações, renderização condicional
utils/routes.test.tsx	Routing	Navegação entre páginas, redirecionamento sem auth
utils/sync.test.tsx	Sincronismo	Pipeline de APIs, logs de progresso, dados retornados

Scripts de Teste

```
npm test
```

Executa todos os testes uma vez e exibe resultado.

```
npm run test:watch
```

Modo watch — re-executa ao salvar arquivos (dev).

9. Deploy e Infraestrutura

Firestore Hosting

O projeto é hospedado no **Firestore Hosting** com CDN global automático. Project ID: **uni0dontopassos**.

Configuração	Valor
Public directory	dist/ (saída do Vite)
SPA Rewrite	/** → /index.html (sem SSR)
Cache JS/CSS/TS	max-age=31536000 (1 ano, imutável)
Cache HTML	no-cache (sempre valida no servidor)
URL pública	https://uni0dontopassos.web.app

Processo de Build e Deploy

- npm run build — compila TypeScript + empacota com Vite → dist/
- firebase deploy — faz upload de dist/ para Firestore CDN
- Assets com hash no nome recebem cache de 1 ano (imutável)
- index.html sem cache garante sempre a versão mais recente do app

Android / Capacitor

Configuração	Valor
App ID	com.dashboard.app
App Name	App Dashboard
Web Dir	dist/ (mesmo build do Vite)
Versão Capacitor	8.3.4
Plataforma	Android (pasta android/)

- npx cap sync — copia dist/ para android/app/src/main/assets/public/
- npx cap open android — abre Android Studio para build nativo
- npx cap run android — executa diretamente em dispositivo/emulador

10. Design System

Paleta de Cores

Token	Hex	Uso Principal
brand-primary	#D81B60	Cor principal — botões, destaques, ícones ativos
brand-secondary	#E91E63	Variação de destaque e hover states
brand-dark	#2D0A1B	Fundo de componentes dark (charts, sidebar night)
brand-bg	#F8F9FA	Background geral em light mode
TABLE_HEADER	#880E4F	Cabeçalho de tabelas
GRAY_DARK	#374151	Textos principais
GRAY_MID	#6B7280	Textos secundários / rótulos
GRAY_LIGHT	#F3F4F6	Fundos alternados de tabela

Tipografia

Fonte principal: **Inter** (sans-serif). Escalas principais:

Uso	Tamanho	Peso
Títulos de seção (KPI)	30px	Bold
Métricas principais	28–30px	Bold
Rótulos de KPI	10–11px	Regular
Corpo de texto	14–16px	Regular
Labels de tabela	12px	Medium

Breakpoints Responsivos

Breakpoint	Largura	Comportamento
xs (custom)	375px	Dispositivos pequenos (iPhone SE)
sm	640px	Smartphones maiores
md	768px	Tablets — troca mobile→desktop layout
lg	1024px	Notebooks
xl / 2xl	1280px / 1536px	Desktops grandes

Acessibilidade

- Focus trap em modais (Sidebar help/logout) — TAB capturado dentro do modal.
- Tecla Escape fecha todos os modais abertos.
- ARIA labels em botões e elementos de navegação.

- Touch targets mínimos de 44px (Tailwind min-h-touch).
- Contraste de cores validado contra WCAG AA.
- Texto responsivo escalável em todos os breakpoints.

11. Usuários e Segurança

A versão atual utiliza **autenticação local via localStorage**, adequada para uso interno controlado. Não há backend, banco de dados externo ou transmissão de credenciais por rede.

Limitações Atuais

- Senhas armazenadas em texto plano no localStorage (sem hash).
- Sem expiração de sessão automática.
- Sem autenticação multifator (MFA).
- Dados de todos os usuários acessíveis por qualquer usuário logado via devtools.
- Sem controle de permissões granular por role (todos acessam todas as páginas).

Recomendações para Produção

- Migrar autenticação para Firebase Authentication (Google/Email+Senha com hash bcrypt).
- Implementar controle de acesso baseado em role (RBAC) por página/funcionalidade.
- Adicionar expiração de sessão (ex.: 8h de inatividade).
- Mover dados sensíveis para Firestore (com regras de segurança).
- Habilitar HTTPS obrigatório (Firebase já força em produção).
- Auditoria de acesso — log de ações críticas (exportar PDF, upload, excluir usuário).

12. Roadmap e Pendências

Próximas Implementações (v1.3+)

Prioridade	Feature	Justificativa
Alta	Firebase Authentication	Substituir localStorage auth por auth real com hash de senhas
Alta	Firestore Integration	Persistência de dados em nuvem multi-usuário em tempo real
Alta	RBAC por Role	Diretores: leitura; Gerência: + upload; Admin: + configurações
Média	API Real Google Ads	Substituir pipeline simulado por integração com Google Ads API v18
Média	API Real Meta ADS	Integração com Meta Marketing API para métricas de campanhas reais
Média	Notificações Push	Alertas de KPIs críticos (churn alto, ROI abaixo da meta)
Baixa	Exportação Excel	Relatórios em .xlsx além de PDF
Baixa	Dashboard iOS	Compilação Capacitor para iOS além de Android
Baixa	Internacionalização	Suporte a inglês para eventuais auditores externos

Dívidas Técnicas Conhecidas

- Arquivo mockData.ts deletado (D src/data/mockData.ts) — verificar ausência de imports órfãos.
- Arquivo reportsData.ts deletado — garantir que Reports.tsx usa apenas mockReports.ts.
- docs/technical-documentation.md não commitado — integrar ao versionamento.
- src/utils/dashboardCalculations.ts, formatters.ts, subMappers.ts — arquivos novos não commitados.
- tsconfig.json e vite.config.ts com modificações pendentes — revisar antes do próximo deploy.
- Autenticação local sem hash — risco de segurança para qualquer ambiente com acesso externo.

Status Git Atual (Jun/2026)

Arquivo	Status
src/App.tsx	Modificado (M)
src/components/layout/Sidebar.tsx	Modificado (M)
src/context/DashboardContext.tsx	Modificado (M)
src/data/mockDashboardData.ts	Modificado (M)
src/data/mockData.ts	Deletado (D)
src/data/reportsData.ts	Deletado (D)
src/pages/DataUpload.tsx	Modificado (M)
src/pages/Login.tsx	Modificado (M)
src/pages/Reports.tsx	Modificado (M)
src/pages/Settings.tsx	Modificado (M)
src/utils/dashboardCalculations.ts	Novo (não rastreado)

src/utls/formatters.ts	Novo (não rastreado)
src/utls/subMappers.ts	Novo (não rastreado)
docs/technical-documentation.md	Novo (não rastreado)

Documento gerado automaticamente em 01/06/2026 às 12:16 · FerTaise Tech · Dashboard Uniodonto Passos v1.2